Compile Time vs. Runtime

Elixir is a compiled language, so your code is transformed before it executes. (Mix handles compilation for us, and files are only recompiled when they change.) Elixir's compiler reads your files from top to bottom to construct the modules and functions that are used in your application.

This is a big departure from Ruby, where code is evaluated and executed at runtime. Many new Elixir developers get confused by what is compile time and what is runtime. The difference matters because you can introduce subtle bugs—or absolute show-stoppers—without being able to identify where the problem is. This section will equip you with just enough to minimize that confusion.

The simplest explanation is everything inside of a function is executed at runtime, and everything outside of a function is executed during compile time.

Let's demonstrate this with an example. Type the following code at lib/examples/modules/compile.ex:

elixir_examples/lib/examples/modules/compile.ex

```
Line 1  defmodule Examples.Modules.Compile do
        IO.puts "I'm at compile-time"
        
        @now Time.utc_now()
      5
        def runtime do
          IO.puts "This was compiled at #{@now}. It is #{runtime_now()}"
        end
        
     10 defp runtime_now do
          Time.utc_now()
        end
      end
```

The @now statement on line 4 is called a module attribute. Module attributes are set during the compilation phase and hold a value or list of values. They are used like read-only variables inside of your module.

When you run this example, note the compile-time message and the differences in timestamps.

```
$ iex -S mix
Compiling 1 file (.ex)
I'm at compile-time
```

```
iex> Examples.Modules.Compile.runtime()
This was compiled at 20:26:17.999151. It is 20:26:19.832422
```